

Task Management System

A Java task tracker for software teams, built around two design patterns.

SEN3006

Software Architecture

June 2026

A. Malak ·

domain, Strategy, SOLID, demo narration

M. Jendawy ·

architecture, Factory Method, lifecycle, demo driver

What we will cover

#	Topic	Who
1	The task tracker problem in plain English	A. Malak
2	What we actually built	A. Malak
3	Architecture in one diagram	M. Jendawy
4	Factory Method, and why we used it	M. Jendawy
5	Strategy, and why we used it	A. Malak
6	Lifecycle as a state machine	M. Jendawy
7	Live demo	J drives, M narrates
8	Extending the system	A. Malak
9	SOLID, fast	A. Malak
10	Reflection and questions	Both

1. The problem

Two things every team task tracker has to do

Tasks are not all the same shape

A bug has a severity and steps to reproduce. A feature has an estimated effort in hours and a business value rating. A documentation task has a document type and a target audience. They share a `Task` interface, but the constructors are genuinely different.

```
new BugTask(title, priority, "MEDIUM", "");  
new FeatureTask(title, priority, 8, 5);  
new DocumentationTask(title, priority, "API", "Developers");
```

Every place that creates tasks (the console menu and the scripted demo) would end up naming concrete classes. Adding a `ResearchTask` later means hunting all of those down.

Creational problem. This is where Factory Method earns its keep.

The same list needs different orderings

Sprint planning wants the list sorted by deadline. Incident response wants it by severity, with bugs first. The daily standup wants it by raw priority. Same list, three contexts, three orderings.

```
switch (sortMode) {  
    case DEADLINE: ... 50 lines ...  
    case SEVERITY: ... 50 lines ...  
    case URGENT:    ... 50 lines ...  
}
```

Every new ordering rule edits the manager. That breaks Open/Closed the moment a fourth context appears.

Behavioral problem. This is where Strategy earns its keep.

2. What we built

One Java engine, two ways to drive it from the terminal

The application

Engine

A single `TaskManager` holds the task list, the active sort strategy, and a registry of factories keyed by task type.

How we run it (terminal application)

1. `java -cp bin Main` runs five self-checking test sections from the command line. This is what the professor asked for.
2. `java -cp bin TaskManagementApp` opens an interactive console menu, also from the terminal.

Numbers

- 16 Java files
- 3 task types
- 3 sort strategies
- 5 lifecycle states

The class diagram

4. Factory Method

Why we picked it for `Task` creation

We never name the task type directly

A bug, a feature, and a documentation task all carry different fields and different defaults. If the console code, the scripted demo, and the test code each wrote `new BugTask(...)` directly, adding a new task type later would mean hunting down every one of those spots and editing them. Instead, the manager keeps a small registry keyed by a string like `"BUG"`, and the call site just says "make me a `BUG`" without knowing which class that is.

Adding a Research task tomorrow is two new files plus one line of registration. Nothing the entry points already wrote has to change.

```
manager.createTask("BUG", "Login broken", "OAuth callback fails", 8);  
manager.createTask("FEATURE", "Dark mode", "Top user request", 5);
```

Two design choices we want to flag

Why is `TaskFactory` an abstract class and not just an interface?

Because each task type has its own defaults. A bug defaults to medium severity. A feature defaults to 8 hours of effort and a business value of 5. A documentation task defaults to API docs aimed at developers. Putting those defaults in each subclass keeps them next to the type they describe, instead of scattering them into a static helper or pushing them onto every caller.

Why is `createTaskWithDeadline(...)` already written on the base class?

Because every task type needs a "create with a due date" variant, and we did not want to write it three times. The base class wraps the abstract `createTask(...)` call and attaches the deadline once. Subclasses only fill in the type-specific part. That is the Template Method shape sitting on top of Factory Method, and it is why a new variant later means editing one base class, not three.

5. Strategy

Why we picked it for sorting

The same list, sorted three different ways

The same list of tasks needs different sorts in different moments. Sprint planning sorts by deadline. An incident sorts by severity, with bugs at the top. The daily standup sorts by raw priority. Without Strategy, the manager would carry one big switch statement covering all three, and we would have to edit it every time a new sort appeared. With Strategy, each sort is a swappable object, and changing the order is a single setter call. A fourth ordering tomorrow is one new file, zero edits to the manager.

```
manager.setPriorityStrategy(new SeverityFirstStrategy());  
// Next call to getPrioritizedTasks() uses the new sort.
```

Two design choices we want to flag

Why not just use Java's built-in `Comparator` ?

Because a name like `PriorityStrategy` tells the reader this is a swappable policy of the system, not a stray comparator buried in a utility class. And one of our strategies, `SeverityFirstStrategy`, does more than pairwise compare. It partitions bugs to the top of the list first, then orders the rest by priority. That partition-then-sort shape is awkward to express through a raw `Comparator`, which only knows how to look at two items at a time.

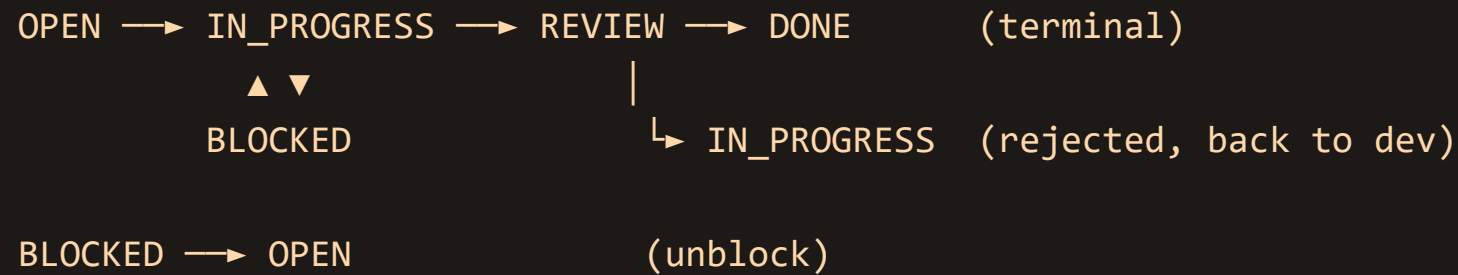
Why is the strategy a setter, not a constructor argument?

Because users change the sort while the app is running. If we passed the strategy into the manager at construction, picking a new sort would mean rebuilding the manager and copying every task into the new instance. One field plus one setter avoids all of that. The trade-off is that the manager is mutable on this one field, which we accepted because the alternative is worse.

6. Lifecycle

A third pattern, almost for free

TaskStatus is its own state machine



Each enum constant declares its allowed next states through `canTransitionTo(...)`.

`AbstractTask.setStatus(...)` checks that method and throws `IllegalArgumentException` on illegal moves.

The State pattern, expressed in a single enum file. No extra classes, no scattered conditionals.

7. Live demo

About three minutes

What we will show in the demo

1. **Scripted self-checks.** `java -cp bin Main`. Five test sections run, each ending in `[PASS]`. Pause on Test 1 (Factory Method) and Test 2 (Strategy) to call out what is happening.
2. **Interactive console.** `java -cp bin TaskManagementApp`. Open the menu, create a `BUG` task, create a `FEATURE` task, switch the sort strategy from Urgent First to Severity First, print the list.
3. **Swap the strategy live.** Inside the interactive console, type the menu option to change the sort and pick `DeadlineFirstStrategy`. Print the list again. The order changes in front of you, the manager itself was never recompiled.
4. **Trigger a factory error.** Type the option to create a task with type `"Foo"`. The engine throws, the console prints the exact reason.
5. **Trigger an illegal transition.** Pick an `OPEN` task and try to mark it `DONE` directly. The engine refuses with the transition that was attempted.

8. Extending it

What changes if you add a new task type or sort rule

Two "what if" cases

Add a `ResearchTask`

```
class ResearchTask extends AbstractTask { ... }

class ResearchTaskFactory
    extends TaskFactory { ... }

manager.registerFactory(
    "RESEARCH", new ResearchTaskFactory());
```

Zero edits to TaskManager, strategies, or other factories. The console menu picks it up automatically because it iterates the registry.

Add a `CheapestFirstStrategy`

```
class CheapestFirstStrategy
    implements PriorityStrategy { ... }

manager.setPriorityStrategy(
    new CheapestFirstStrategy());
```

Zero edits to TaskManager or existing strategies. Add it to the console menu to expose it to the user.

Why this matters

- Open/Closed in literal numbers.
- New features without touching tested code.
- Test 5 in `Main.java` proves the same property at runtime.

How the design lines up with SOLID

- **S, Single Responsibility.** Factories build, strategies sort, the manager coordinates. One job each.
- **O, Open/Closed.** New task type, new sort rule. Neither edits existing classes.
- **L, Liskov.** Every factory subclass and every strategy implementation can stand in for its abstraction without surprises.
- **I, Interface Segregation.** `PriorityStrategy` has one method. The `Task` interface is minimal.
- **D, Dependency Inversion.** `TaskManager` references only abstractions. There is no `new BugTask(...)` inside the manager.

9. Wrapping up

What worked, what we would do next

Honest reflection

What worked

- The factory registry meant every entry point auto-discovered new task types. We confirmed this when we added Documentation late in the project.
- The Strategy interface let us prototype four orderings in one afternoon and then keep the best three.
- The lifecycle enum caught two real bugs during development that would have shipped otherwise.
- The scripted `Main` doubled as a test harness without us pulling in JUnit.

What we would do next

- Persistence. Everything currently lives in memory.
- Tag based filtering with a Specification pattern. The console already filters by status, tags would be the next step.
- Async dispatch for long running operations.
- A small REST layer on top of the same engine.

Questions?

Thank you for listening.

A. Malak M. Jendawy